

How to exploit vulnerabilities

Tips and tricks for tackling Assignment 2 of the Security course

Francesco Bariatti

Security course, LIACS, 2025/26

In this document we give some tips and trick on which steps an “attacker” can take in order to verify a vulnerability of a program and explore it in order to eventually exploit it to reach their goal. These tips will be useful when you attempt Assignment 2 (in particular the “hacker” track), as well as to understand the concepts of software security that are presented in the course.

Disclaimer. The presence of a vulnerability in a software does not inherently authorize you to exploit the software! Not even for “learning” or “finding vulnerabilities”! The information that we present in this document is meant to help you get an understanding of software security in the context of the Security course at LIACS. Even in the context of this course, you are only authorized to attack the programs for which such authorization is given explicitly in the assignment(s).

1 Exploiting a vulnerability in general

The main difficulty that we observed for student trying to tackle our assignments, is trying to do *everything at once*. While you might have an idea on which vulnerability is present, and how such vulnerability could be exploited *in theory*, achieving that on a single try is unlikely.

In reality, an attacker might want to limit the time that they spend on a vulnerable system in order not to be detected, or to avoid breaking the system in a way that blocks them from attempting further exploits. However, that is not the case for you in the context of the course assignments. You are given an environment in which you can try multiple times to go through each single program. Therefore, the first tip for successfully completing the assignment is to *have patience*, and *take one step at a time*. More so, after every step you should stop and analyze what you learned and what do you think that is happening inside the program.

Without going in the details of each possible vulnerability that you will encounter, we suggest that you take the following steps for each:

1. Observe the program that you need to attack. Try to interact with it in the “expected” way: what is the program supposed to do? What is the expected behaviour of the program? What might the program *assume* about its environment and what you provide it with? Read through the code of the program (if that is available) or try to infer the code from other indicators (e.g. disassembling the program, listing files in its directory, etc).

Thanks to this analysis, you might come up with some idea on where a vulnerability lies, and in which way the program is not protected against user action. For instance, you might find that the program does not check for the maximum length of the input that you provide it with.

2. Try to “exploit” this possible vulnerability with some arbitrary payload or “attack”. The goal in this step is to alter the normal behaviour of the program, in order to verify that you can indeed affect the normal execution of the program with your payload. You will probably not be able to complete the full exploit at this step, but an unexpected error message or a change in how the program behaves can tell you that you manage to affect it in a way that the author’s did not expect.

For instance if you think that the program does not limit the size of your input, try to pass an input that is a hundred times the letter “A”. This will probably not be exactly what you need to pass, but the program might crash with a segmentation fault error: this indicates that the program encountered a situation that was not expected. Or it might print “AAAAA[...]: command not found”: this shows that the program attempted to use your payload as some sort of command. In both these cases you affected the behaviour of the program. That means that you probably have found an entry point for attacking the program.

3. You now know that you can take some action to affect the program behaviour. The goal is now to find *which* actions/inputs makes the program behave the way that *you* want. This step is probably the most complicated. Think about *how* the actions that you took in the previous step affected the program. Try to execute the program in your head: what is happening and what would you rather be happening? Your goal is now to tinker bit by bit to eventually direct the program towards your goal.

This step can very well have many sub-steps. Maybe your goal is to get the program to execute function *b* instead of function *a*. Then you can proceed as follows:

- (a) Get the program to execute a function *x* that does not exist. The program should crash.
- (b) Get the program to execute some functions *c* and *d* that exist but are not your goal. This might be easier than executing *b*.
- (c) Finally gather everything that you learned and get the program to actually execute *b*.

Again, you should proceed by little steps that you understand. If you try to immediately get to your goal you will probably fail, but with a failure that does not teach you anything. If instead you take small steps, each failure will teach you something more about the next steps that you need to take.

4. If you did everything correctly, you have now successfully exploited a program. Congratulations! You should now profit from your exploits. In the context of the assignment this means completing the exercise, and improving your grade!
5. An additional step whenever you manage to complete your exploit is to gather what you have learned. It can often happen that you try many different things and eventually one surprisingly works, but you are not sure why. An important final step is to understand exactly what happened.

You should be sure on how you achieved the exploit, and *be able to achieve that again*.

2 Exploiting a buffer overflow: jumping to attacker-injected code

While the previous section explains the steps that you can take to exploit any vulnerability in general, which can therefore also be applied to buffer overflow, in this section we give more details about how to exploit a buffer overflow vulnerability in particular.

Note that in this section we go over the most complete setup for a buffer overflow, in which we inject our own executable code and jump to it. As covered in the lectures, you might sometimes have simpler situations, where overwriting the content of a single variable might be enough to achieve your goal.

For the hacker-track levels containing buffer overflow vulnerabilities, it is highly encouraged that you inspect the program that you are trying to attack with GDB: set some breakpoints and inspect the contents of memory and registers at critical points, in order to understand what is happening inside the program.

1. If you are unsure on how a program works and how it jumps between function calls, run the program normally through GDB. Put breakpoints before and after function calls, step through one instruction at a time, and inspect the content of all registers and the contents of the stack. Try to identify how the `rbp` register is used, and where the values of the previous `rbp` and `rip` (i.e. the return address) of a function are stored.
2. Find a vulnerable buffer. A vulnerable buffer is a variable whose content is entered via a vulnerable function, i.e. a function that does not check what the maximum size of the buffer is w.r.t. the values entered. Generally in a buffer overflow exploit we aim to overwrite the return address of a function, as that is stored after all the function's local variables on the stack. Note that a buffer overflow can only affect the contents of the memory *after* the end of the buffer. So if in a function's stack you have the variables `a`, `buffer`, `b`, an overflow of `buffer` can only affect `b` and everything beyond `b` (i.e. the return address of the function). This also means that if the buffer is declared in function `f` you can only affect the return address of `f`, even if the vulnerability happens in another function that is called by `f`.
3. Find the address of buffer w.r.t. `rbp`. You can infer that by disassembling the program's code and checking for function calls that use the buffer, as the buffer's address will be passed as an argument. For instance if you see the instructions `lea rdi [rbp-0x12]` and `call function_that_uses_buffer` you can deduce that the buffer starts 18 bytes (0x12) before `rbp`. Since functions can take multiple parameters, you might have to sort through multiple instructions and variables to determine the correct address of the buffer.

4. Try to overflow the buffer with some arbitrary data that is long enough to overflow the saved `rbp` and `rip`. For instance, if you know that the buffer starts 18 bytes before `rbp`, you can try to pass as input $18 + 8 + 8 = 34$ bytes at least. Use a *known value*. For instance pass 34 times the byte `0x42` (any arbitrary value works). Use GDB to visualize the content of memory, as well as the saved values of `rbp` and `rip`. If you succeeded they should be all `0x4242424242424242`. If that is the case, that means that you managed to affect the program status and you just executed a buffer overflow!
5. Continue executing the program until just before the `ret` instruction. Then step over a single instruction. You should see that your program tries to jump to address `0x4242424242424242` and then immediately crashes (as nothing is probably stored at that address). Now change your payload so that the program instead jumps to an address that you decide. Try `0x1122334455667788`, as well as `0x1234567812345678`. This is not straightforward the first time you try because of something called “endianness”. Once you manage to make the program jump to any address that you want you are ready for the next step: putting something interesting at that address!
6. Now you need to make sure that you load into memory some useful code, for instance the code that will give you access to a shell. You can craft your own assembly code or find one online (or some of the simpler levels in the assignment might already have useful code somewhere for you to use). Modify your payload so that the correct binary instructions are inserted into memory. Take care of inspecting memory contents with GDB and the `x/i` command, to verify that the injected code is actually what you expect. This step can be tricky and very often an exploit that does not work is due to incorrect code being executed.
7. Once you have finalised the payload, make sure that you can still reliably choose the address at which the program jumps. Very often when trying different payloads you will end up shifting your code by some bytes, and the part that inject the new address might become misaligned.
8. Find the address of your payload and make the program jump to it. Remember that using a NOP sled will make it much simpler to guess a good address! Stop the program as soon as it jumps and execute instruction by instruction to make sure that your code is executed as you expect.
9. The goal is now to execute the attack outside of GDB! When you debug a program that has the `setgid` bit set, that bit is ignored (otherwise attacking them would be trivial), so your attack must work outside of a debugger. Read the following sections on how to do that, as well as some critical parts that make it more difficult.

2.1 Creating binary payloads with python

In order to execute buffer overflow attacks, we very often have to give some binary input to programs. For instance a program might ask for our name, made

of letters, but instead we want to give it a very long and complex sequence of non-letter bytes. In python it is possible to print bytes `0x1234` to the standard output by using `sys.stdout.buffer.write(b"\x12\x34")`. Using python allows you to write more readable programs and not get lost in the different byte values.

What you can do in order to generate a payload with python is to write a script that writes bytes to the output as explained above, and then call it as `python3 myscript.py > payload.bin`. Then, you can use the resulting binary file as follows:

- Inside gdb: start your program with `run < payload.bin`
- Outside gdb: execute `(cat payload.bin ; sleep 0.1 ; cat - )| /vulnerable_program`. This first prints the content of the payload to the program inputs, and prints whatever you type to the input (which allows you to enter commands in the shell if you manage to open one with your attack).

We suggest using the fact that you can write full python scripts as an opportunity to use insightful comments and variable names, so as not to get lost in the payload that you need to write.

Other useful things for writing binary in python:

- If you need to repeat a byte, you can use `b"\x12" * 40` (this generates 40 bytes each with value `0x12`).
- If you need to convert an integer `x` to bytes you can use `x.to_bytes()`. Check out the parameters for that function!

2.2 Tricky differences between running inside and outside GDB

When executing a buffer overflow attack, you rely on guessing addresses of values stored in the program's stack. Often these will come from you observing a program running inside gdb. However, a difficulty in executing this attacks is that the stack can differ between when you execute the program inside gdb and outside. This is due to the fact that when a program is started, all program arguments as well as environment variables are put on the stack. Therefore if arguments and environment variables differ between two situations, the stack will not start at the same address, shifting all subsequent addresses even for similar executions!

There are two main reasons why the stack might differ between inside and outside GDB:

- GDB sets two environment variables named `LINES` and `COLUMNS`, which are often not present in your normal environment. You can remove them with `unset env LINES` and `unset env COLUMNS`.
 - You can use the `show environment` command inside gdb and the `printenv` command outside of gdb to show all environment variables. If other variables differ, you might want to fix them as well.

- GDB *always* calls the program that you execute with its full path. So even if you start gdb as `gdb ./level1`, gdb will call `/levels/level1/level1` (assuming that is the full path). Since the path to a program ends up being its very first argument –which is stored on the stack– this difference can be very impactful w.r.t. stack addresses.

Similarly, addresses can differ between users because user-specific information can be stored in the environment variables. So an address that works for your colleague might not work directly for you.

3 Conclusion

The main message of this document is that you should *take your time* and *take one step at a time* when trying to explore vulnerable programs and exploit vulnerabilities for the course assignments. If you try to everything at once or change too many things at once, there is a high chance that your attempt will not work, while at the same time not giving you any information on what went wrong.

Working on the assignments is as much a discovery on how vulnerabilities work as a discovery on how programs normally work (before you attack them), which is fundamental knowledge for a computer scientist!